

NP-Complete Problems

Question-Answer Notes for Exercises 34.5-1 to 34.5-8

Background

Proof Pattern

To prove a decision problem is **NP**-complete, show it is in **NP** and then reduce a known **NP**-complete problem to it. In these exercises, the source problems are usually CLIQUE, 3-CNF-SAT, SUBSET-SUM, or HAM-CYCLE.

1 Exercise 34.5-1

Question 34.5-1

The subgraph-isomorphism problem takes two undirected graphs G_1 and G_2 , and asks whether G_1 is isomorphic to a subgraph of G_2 . Show that SUBGRAPH-ISOMORPHISM is **NP**-complete.

Answer

SUBGRAPH-ISOMORPHISM is in **NP** because a certificate is an injective map from $V(G_1)$ into $V(G_2)$.

Proof

Given an injection $f : V(G_1) \rightarrow V(G_2)$, the verifier checks that for every edge $(u, v) \in E(G_1)$, the edge $(f(u), f(v))$ appears in G_2 . This is polynomial time, so SUBGRAPH-ISOMORPHISM belongs to **NP**.

For NP-hardness, reduce

$$\text{CLIQUE} \leq_p \text{SUBGRAPH-ISOMORPHISM}.$$

Given a CLIQUE instance $\langle G, k \rangle$, let

$$G_1 = K_k \quad \text{and} \quad G_2 = G.$$

Then K_k is isomorphic to a subgraph of G iff G contains k vertices with all possible edges between them, which is exactly a clique of size k .

The construction is polynomial. Since CLIQUE is **NP**-complete and SUBGRAPH-ISOMORPHISM is in **NP**, SUBGRAPH-ISOMORPHISM is **NP**-complete.

2 Exercise 34.5-2

Question 34.5-2

Given an integer $m \times n$ matrix A and an integer m -vector b , the 0-1 integer-programming problem asks whether there exists a vector $x \in \{0, 1\}^n$ such that

$$Ax \leq b.$$

Prove that 0-1 integer programming is **NP**-complete.

Answer

Reduce from 3-CNF-SAT. Each Boolean variable becomes a 0-1 variable, and each clause becomes one linear inequality.

Membership in NP

A certificate is a vector $x \in \{0, 1\}^n$. The verifier computes Ax and checks all inequalities $Ax \leq b$ in polynomial time. Thus 0-1-IP is in **NP**.

Reduction from 3-CNF-SAT

Let ϕ be a 3-CNF formula with variables $x_1, \dots, x_n$ and clauses $C_1, \dots, C_m$. For a Boolean variable x_i , use the integer variable $x_i \in \{0, 1\}$, where 1 means true and 0 means false. For a clause C_j , let P_j be the set of indices of variables appearing positively in C_j , and let N_j be the set of indices of variables appearing negated in C_j . The clause is satisfied iff

$$\sum_{i \in P_j} x_i + \sum_{i \in N_j} (1 - x_i) \geq 1.$$

Rewrite this as a $\leq$ inequality:

$$-\sum_{i \in P_j} x_i + \sum_{i \in N_j} x_i \leq |N_j| - 1.$$

This gives one row of A and one entry of b for each clause.

The formula ϕ is satisfiable iff the resulting 0-1 integer program is feasible. The construction is polynomial. Therefore 0-1-IP is **NP**-hard, and since it is in **NP**, it is **NP**-complete.

3 Exercise 34.5-3

Question 34.5-3

Integer linear programming is like 0-1 integer programming, except that the variables may be any integers. Assuming 0-1 integer programming is **NP**-hard, show that integer linear programming is **NP**-complete.

Answer

Force each integer variable to lie between 0 and 1.

Reduction

Given a 0-1 integer-programming instance

$$Ax \leq b, \quad x_i \in \{0, 1\},$$

construct an integer linear program with the same variables, but add the constraints

$$x_i \leq 1 \quad \text{and} \quad -x_i \leq 0$$

for every i . Equivalently, use

$$\begin{pmatrix} A \\ I \\ -I \end{pmatrix} x \leq \begin{pmatrix} b \\ \mathbf{1} \\ \mathbf{0} \end{pmatrix}.$$

Since the variables are required to be integers, these inequalities force

$$x_i \in \{0, 1\}.$$

Thus the new ILP instance is feasible iff the original 0-1 instance is feasible. The reduction is polynomial, so ILP is **NP**-hard.

ILP is in **NP** because a feasible integer linear system has a feasible integer solution with polynomially many bits, and such a solution can be verified by substitution. Therefore ILP is **NP**-complete.

4 Exercise 34.5-4

Question 34.5-4

Show how to solve SUBSET-SUM in polynomial time if the target value t is expressed in unary.

Answer

Use the standard dynamic programming algorithm for subset sum.

Algorithm

Let

$$S = \{a_1, a_2, \dots, a_n\}.$$

Ignore any $a_i > t$, since positive numbers larger than t cannot be used in a subset summing to t . Define a Boolean table

$$\text{dp}[i, s]$$

to mean: using the first i numbers, can we form sum s ? Initialize

$$\text{dp}[0, 0] = \text{true}$$

and $\text{dp}[0, s] = \text{false}$ for $s > 0$. The recurrence is

$$\text{dp}[i, s] = \text{dp}[i-1, s] \vee (s \geq a_i \text{ and } \text{dp}[i-1, s-a_i]).$$

The answer is $\text{dp}[n, t]$.

The table has $(n+1)(t+1)$ entries, so the running time is

$$\mathcal{O}(nt).$$

If t is written in unary, then t is at most linear in the input length. Thus this algorithm is polynomial time in the input length.

5 Exercise 34.5-5

Question 34.5-5

The set-partition problem asks whether a set S of numbers can be partitioned into two sets A and $\bar{A} = S - A$ such that

$$\sum_{x \in A} x = \sum_{x \in \bar{A}} x.$$

Show that SET-PARTITION is **NP**-complete.

Answer

Reduce from SUBSET-SUM.

Membership in NP

A certificate is one side A of the proposed partition. The verifier sums the numbers in A and in $S - A$ and checks equality. This is polynomial time.

Reduction from SUBSET-SUM

Given a SUBSET-SUM instance (S, t) , let

$$x = \sum_{s \in S} s.$$

If $t > x$, map to a fixed no-instance of SET-PARTITION, such as $\{1, 2\}$. Otherwise construct

$$S' = S \cup \{x + t, 2x - t\}.$$

The total sum of S' is

$$x + (x + t) + (2x - t) = 4x,$$

so each side of an equal partition must sum to $2x$.

If some subset $B \subseteq S$ has sum t , then

$$B \cup \{2x - t\}$$

has sum $2x$, and its complement also has sum $2x$. Thus S' is partitionable.

Conversely, suppose S' has an equal partition. The two added numbers cannot lie on the same side because their sum is $3x > 2x$. Therefore they lie on opposite sides. The side containing $2x - t$ must be completed by original elements of total

$$2x - (2x - t) = t.$$

So S has a subset summing to t .

Thus

$$(S, t) \in \text{SUBSET-SUM} \iff S' \in \text{SET-PARTITION}.$$

The construction is polynomial. Since SUBSET-SUM is **NP**-complete and SET-PARTITION is in **NP**, SET-PARTITION is **NP**-complete.

6 Exercise 34.5-6

Question 34.5-6

Show that the Hamiltonian-path problem is **NP**-complete.

Answer

Reduce from HAM-CYCLE by splitting one vertex into two endpoints.

Membership in NP

A certificate is a list of all vertices in path order. The verifier checks that each vertex appears exactly once and that consecutive vertices are adjacent. Thus HAM-PATH is in **NP**.

Reduction from HAM-CYCLE

Given a graph $G = (V, E)$, choose a vertex $v \in V$. Construct G' as follows:

- replace v by two vertices v_{in} and v_{out} ;
- for every edge $(v, u) \in E$, add edges (v_{in}, u) and (v_{out}, u) ;
- keep all edges of G not incident on v ;
- add two leaves s and t with edges (s, v_{in}) and (v_{out}, t) .

If G has a Hamiltonian cycle

$$v, u_1, u_2, \dots, u_{n-1}, v,$$

then G' has the Hamiltonian path

$$s, v_{\text{in}}, u_1, u_2, \dots, u_{n-1}, v_{\text{out}}, t.$$

Conversely, any Hamiltonian path in G' must start and end at the leaves s and t . Removing s and t gives a path from v_{in} to v_{out} through every original vertex once. Replacing v_{in} and v_{out} by the single vertex v closes this path into a Hamiltonian cycle of G .

Thus

$$G \in \text{HAM-CYCLE} \iff G' \in \text{HAM-PATH}.$$

The construction is polynomial. Since HAM-CYCLE is **NP**-complete, HAM-PATH is **NP**-hard, and since it is in **NP**, it is **NP**-complete.

7 Exercise 34.5-7

Question 34.5-7

The longest-simple-cycle problem asks for a simple cycle of maximum length. Formulate a related decision problem and show that it is **NP**-complete.

Answer

Use the decision problem

$$\text{LONG-SIMPLE-CYCLE} = \{\langle G, k \rangle \mid G \text{ has a simple cycle of length at least } k\}.$$

Proof

LONG-SIMPLE-CYCLE is in **NP** because a certificate is the claimed cycle. The verifier checks that the vertices are distinct, that consecutive vertices are adjacent, that the last returns to the first, and that the length is at least k .

For NP-hardness, reduce

$$\text{HAM-CYCLE} \leq_p \text{LONG-SIMPLE-CYCLE}.$$

Given a graph $G = (V, E)$, output

$$\langle G, |V| \rangle.$$

Then G has a simple cycle of length at least $|V|$ iff it has a simple cycle using all vertices, i.e., a Hamiltonian cycle.

The construction is polynomial. Since HAM-CYCLE is **NP**-complete and LONG-SIMPLE-CYCLE is in **NP**, LONG-SIMPLE-CYCLE is **NP**-complete.

8 Exercise 34.5-8**Question 34.5-8**

In the half 3-CNF satisfiability problem, the input is a 3-CNF formula with an even number of clauses. The question is whether some assignment makes exactly half the clauses true and exactly half false. Prove that HALF-3-CNF-SAT is **NP**-complete.

Answer

Reduce from 3-CNF-SAT by adding clauses whose truth values are controlled by one new variable.

Convention

This proof uses the common convention that a 3-CNF clause has three literal positions, so repeated literals such as $(y \vee y \vee y)$ are allowed. If a course requires three distinct literals, these repeated clauses can be replaced by the standard constant-size padding gadgets.

Membership in NP

A certificate is a truth assignment. The verifier evaluates all clauses and counts how many are true. It accepts iff exactly half are true. This is polynomial time.

Reduction from 3-CNF-SAT

Let ϕ be a 3-CNF formula with m clauses. Introduce a new variable y . Let

$$T = (y \vee \neg y \vee y) \quad \text{and} \quad F = (y \vee y \vee y).$$

The clause T is always true, while F has the same truth value as y .

Construct

$$\phi' = \phi \wedge \underbrace{T \wedge \cdots \wedge T}_{m \text{ copies}} \wedge \underbrace{F \wedge \cdots \wedge F}_{2m \text{ copies}}.$$

The formula ϕ' has

$$m + m + 2m = 4m$$

clauses, which is even.

If ϕ is satisfiable, take a satisfying assignment for ϕ and set $y = 0$. Then all m clauses of ϕ are true, all m copies of T are true, and all $2m$ copies of F are false. Hence exactly $2m$ of the $4m$ clauses are true.

Conversely, suppose ϕ' has an assignment making exactly $2m$ clauses true. If $y = 1$, then all m copies of T and all $2m$ copies of F are true, so at least $3m > 2m$ clauses are true, impossible. Thus $y = 0$. Then all $2m$ copies of F are false and all m copies of T are true. To reach exactly $2m$ true clauses, all m original clauses of ϕ must be true. Therefore ϕ is satisfiable.

Thus

$$\phi \in \text{3-CNF-SAT} \iff \phi' \in \text{HALF-3-CNF-SAT}.$$

The construction is polynomial. Since 3-CNF-SAT is **NP**-complete and HALF-3-CNF-SAT is in **NP**, HALF-3-CNF-SAT is **NP**-complete.

Final Summary

Reduction Map

Problem	Reduction idea
SUBGRAPH-ISOMORPHISM	Reduce from CLIQUE using $G_1 = K_k$.
0-1-IP	Reduce from 3-CNF-SAT by turning each clause into one inequality.
ILP	Add $0 \leq x_i \leq 1$ constraints to a 0-1 program.
Unary SUBSET-SUM	Dynamic programming in $\mathcal{O}(nt)$, polynomial when t is unary.
SET-PARTITION	Reduce from SUBSET-SUM by adding $x + t$ and $2x - t$.
HAM-PATH	Split one vertex of a Hamiltonian-cycle instance into two endpoints.
LONG-SIMPLE-CYCLE	Ask whether there is a simple cycle of length at least $ V $.
HALF-3-CNF-SAT	Add always-true clauses and y -controlled clauses.